

Paper___PAKDD_2025_HetGSMOTE.pdf

 National Institute of Science Education and Research

Document Details

Submission ID

trn:oid::3117:410685385

Submission Date

Nov 30, 2024, 5:55 PM GMT+5:30

Download Date

Nov 30, 2024, 6:00 PM GMT+5:30

File Name

Paper___PAKDD_2025_HetGSMOTE.pdf

File Size

407.0 KB

12 Pages

5,444 Words

27,500 Characters

18% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **58 Not Cited or Quoted 18%**
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 16%  Internet sources
- 17%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

-  **Replaced Characters**
13 suspect characters on 4 pages
Letters are swapped with similar characters from another alphabet.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

-  **58** Not Cited or Quoted 18%
Matches with neither in-text citation nor quotation marks
-  **2** Missing Quotations 0%
Matches that are still very similar to source material
-  **0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 16%  Internet sources
- 17%  Publications
- 0%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	arxiv.org	3%
2	Internet	assets.researchsquare.com	2%
3	Internet	www.mdpi.com	1%
4	Internet	www.teses.usp.br	1%
5	Internet	www.iccs-meeting.org	1%
6	Internet	www.arxiv-vanity.com	1%
7	Internet	web.archive.org	1%
8	Internet	ouci.dntb.gov.ua	1%
9	Internet	deepai.org	1%
10	Internet	os.unil.cloud.switch.ch	1%

11	Internet	www.nature.com	0%
12	Internet	mdpi-res.com	0%
13	Publication	"Artificial Neural Networks and Machine Learning – ICANN 2019: Workshop and S...	0%
14	Publication	Jintao Wen, Xianghong Tang, Jianguang Lu. "An imbalanced learning method bas...	0%
15	Publication	Ke Shu, Huifang Ma, Jinpeng Yang, Di Zhang. "GraphSmin: Imbalanced dissolved ...	0%
16	Internet	ebin.pub	0%
17	Publication	"Advances in Information Retrieval", Springer Science and Business Media LLC, 20...	0%
18	Publication	Tianxiang Zhao, Xiang Zhang, Suhang Wang. "GraphSMOTE: Imbalanced Node Cla...	0%
19	Internet	cmsworkshops.com	0%
20	Internet	ojs.aaai.org	0%
21	Publication	"Knowledge Science, Engineering and Management", Springer Science and Busin...	0%
22	Publication	Bei Bi, Zhiwei Zhang, Pengpeng Qiao, Ye Yuan, Guoren Wang. "Chapter 32 FedSig:...	0%
23	Publication	Jian Liu, Lei Gao, Sujie Guo, Rui Ding, Xin Huang, Long Ye, Qinghua Meng, Asef Na...	0%
24	Publication	Liwen Xu, Huaguang Zhu, Jiali Chen. "Imbalanced graph learning via mixed entro...	0%

25	Publication	Liying Zhang, Haihang Sun. "ESA-GCN: An Enhanced Graph-Based Node Classificat...	0%
26	Internet	egusphere.copernicus.org	0%
27	Internet	listens.online	0%
28	Publication	"Web and Big Data", Springer Science and Business Media LLC, 2024	0%
29	Publication	Claiborne, Jerry Craig. "A Machine Learning Approach to Transit Fraud Detection"...	0%
30	Publication	Naeem Seliya, Zhiwei Xu, Taghi M. Khoshgoftaar. "Addressing Class Imbalance in ...	0%
31	Internet	assets-eu.researchsquare.com	0%
32	Internet	dokumen.pub	0%
33	Internet	student.dei.uc.pt	0%
34	Internet	www2.mdpi.com	0%
35	Publication	Zhao Li, Yuying Xing, Jiaming Huang, Haobo Wang, Jianliang Gao, Guoxian Yu. "La...	0%
36	Publication	Run Liu, Chongchong Qi, Teik-Hua Law. "Advances in Traffic Transportation and C...	0%

HetGSMOTE: Improving Imbalance in Heterogeneous Graphs through Oversampling

No Author Given

No Institute Given

Abstract. Imbalanced class distributions are a common challenge in graph datasets, often leading to suboptimal model performance for minority classes. This work introduces HetGSMOTE, a novel oversampling technique tailored to address class imbalance in heterogeneous graphs. Extending the GraphSMOTE method from homogeneous graphs, HetGSMOTE adapts the Synthetic Minority Oversampling Technique (SMOTE) to operate in heterogeneous graph settings. Unlike i.i.d.-type data, graph-based data involves complex relational structures encoded in edges, which vanilla SMOTE fails to preserve. Heterogeneous graphs further complicate this process with their diverse node and edge types, necessitating the integration of contextual information. HetGSMOTE constructs content-aggregated and neighbor-type-aggregated embedding spaces to preserve node similarities within the graph. Additionally, edge generators are trained for different node types to capture and replicate relational structures from the heterogeneous graph. By evaluating various datasets from different domains, we assert HetGSMOTE's versatility and performance improvements over existing methods, particularly in highly imbalanced scenarios.

Keywords: Heterogeneous Graphs · Oversampling · Class Imbalance

1 Introduction

Graph-based learning via Graph Neural Networks (GNNs) have garnered significant attention for modeling intricate relationships and dependencies within structured data. Among graph structures, heterogeneous graphs (HetGs) are ubiquitous, representing various real-world data such as social networks, bibliographic networks, and recommender systems [10,8]. HetGs feature diverse types of entities and relations, highlighting complex relationships, with nodes carrying both structured and unstructured content. Consequently, tasks like node classification [14,12] and link prediction [13] are critical. Heterogeneous Graph Neural Networks (HGNNs) [14] perform well in node classification tasks but are generally designed for balanced class distributions. However, real-world applications often suffer from significant class imbalance, leading to suboptimal performance and necessitating imbalanced learning techniques like oversampling on HetGs. This paper introduces HetGSMOTE, a novel oversampling approach for heterogeneous graphs that leverages Synthetic Minority Oversampling Techniques (SMOTE) on node representation matrices to address class imbalance.

2 No Author Given

While SMOTE has shown success in balancing class distributions in homogeneous graphs [16], its application to HetGs poses unique challenges, such as incorporating node type, edge type, and metapath-based contextual information. HetGSMOTE addresses these by extending the GraphSMOTE [16] framework to HetGs. It constructs a content-aggregated and neighbor-type-aggregated embedding space to encode node similarities, enabling synthetic sample generation while preserving contextual relationships. Additionally, edge generators model relational information between node types, ensuring synthesized samples retain essential structural characteristics. HetGSMOTE mitigates inter-class imbalance by generating synthetic samples for imbalanced classes and avoids the generation of noisy samples by producing safer embedding spaces for oversampling [4]. However, it does not directly address the problem of intra-class imbalances in the dataset [5,2]. The contributions of this paper are threefold: first, we investigate the problem of class imbalance in heterogeneous graphs, a challenge with significant real-world implications that has received limited prior attention. Second, we propose HetGSMOTE, an extension of the GraphSMOTE approach tailored for heterogeneous graphs, incorporating novel strategies for oversampling and edge generation. Lastly, we validate the effectiveness of HetGSMOTE through comprehensive experiments conducted on diverse datasets, showcasing its superior performance compared to baseline methods.

2 Related Works

Class imbalance, where one class significantly outnumbers another, leads to biased models and poor generalization, evident in tasks like fraud detection, rare disease identification, and bot recognition. Addressing this issue involves algorithm-level, data-level, and hybrid strategies [16]. Data-level methods, such as oversampling and data augmentation, increase minority class samples. Algorithm level approaches include cost-sensitive techniques, ensemble learning, and threshold adjustments [1]. Hybrid methods combine these strategies, e.g., classifier-specific models [7]. Oversampling generates synthetic minority samples, with methods like SMOTE, which interpolates between samples. Its improved variants such as Borderline-SMOTE [4], ADASYN [5], and KMeans SMOTE [2] further address intricacies like intra-class imbalance. Oversampling has proven effective in numerous machine learning domains, addressing the issue of limited minority data [7] and improving model performance. HetG learning has evolved from manual feature engineering to representation learning, broadly categorized into shallow and deep models [15]. Shallow models include random walk-based strategies like metapath2vec, which use metapath-guided random walks, and decomposition methods such as HERec [8]. DeepWalk introduced SkipGram embeddings for node co-occurrence probabilities. Variants like Spacey, JUST, and HHNE [10] add enhancements like jump-stay strategies and heterogeneous walks. Deep models leverage Heterogeneous Graph Neural Networks (HGNNs), with unsupervised methods like HetGNN [14], and semi-supervised approaches like HAN [12] using attention mechanisms. Advanced HGNNs, like MAGNN

and HGT [3], refine intra- and inter-metapath aggregations. Techniques such as encoder-decoder models [10] and adversarial frameworks like GraphGAN [11] further enrich HetG representation learning. Oversampling techniques for graph data, like GraphSMOTE [16], address class imbalance in homogeneous graphs by generating synthetic nodes and connections. Other methods, such as Graph-DAO and GraphMixup [9], use latent space sampling and semantic relations. Despite progress, SMOTE has found limited applications in graph-based learning. For HetGs there are even fewer techniques like FincGAN [6], an adversarial GAN-based approach. This motivates our work on the extension of GraphSMOTE to heterogeneous graphs.

3 Problem Statement

Consider a heterogeneous graph, denoted as $\mathcal{H} = (U, \mathcal{A}, \mathcal{B})$. Here, U represents the set of nodes $\{u_1, \dots, u_m\}$, and $U_p \subset U$ denotes a subset of nodes of type p . $\mathcal{A}$ refers to the collection of adjacency matrices, where $\mathcal{A}_{pq} \subset \mathcal{A}$ defines the adjacency matrix for nodes of types p and q . Specifically, $\mathcal{A}_{pq} \in \mathbb{R}^{m_s \times m_t}$, where m_p and m_q represent the number of nodes of types p and q , respectively. The set of attribute matrices, $\mathcal{B}$, includes matrices $\mathcal{B}_j^p \in \mathbb{R}^{m_p \times d}$, where each $\mathcal{B}_j^p$ is the j -th attribute matrix for nodes of type p , and d indicates the embedding dimension. In particular, $\mathcal{B}_j^p[u_i, :] \in \mathbb{R}^{1 \times d}$ is a row vector representing the j -th attribute embedding for node u_i of type p . Class labels for nodes of type p are represented by $Y_p \in \mathbb{R}^{m_p}$. For training, typically only a small subset of labeled nodes $Y'_p \subset Y_p$ is available for a given class, corresponding to nodes $U'_p \subset U_p$. In classification tasks with k distinct classes, represented by $\{c_1, \dots, c_k\}$, we define the size of each class, $|c_i|$, as the number of nodes in class i . The imbalance ratio R_i for each class is given by:

$$R_i = \frac{|c_i|}{\max_j |c_j|}$$

where $\max_j |c_j|$ is the size of the largest class. A perfectly balanced dataset, where each class has the same number of nodes, corresponds to $R_i = 1$ for all classes. Conversely, lower values of R_i indicate a higher degree of imbalance, with some classes having significantly fewer nodes than others. In our study, we utilize a semi-supervised learning approach for node classification on diverse heterogeneous graphs in a transductive framework. While the entire graph structure is accessible during both training and testing, only a small portion of nodes carry labels. These labeled nodes are divided into training, validation, and test subsets to guide the learning process. Given a heterogeneous graph $\mathcal{H}$ with an imbalanced class distribution and a label set Y available only for a subset of nodes, our objective is to train a node classification function f that achieves robust performance across both majority and minority classes, that is: $f(\mathcal{H}) \rightarrow \mathcal{Y}$

4 No Author Given

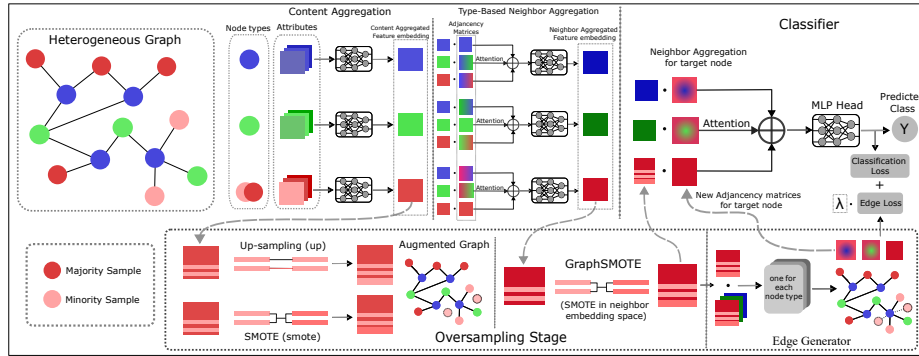


Fig. 1. Pipeline for HetGSMOTE

4 Methodology

As shown in Fig. 1, the pipeline of HetGSMOTE consists of four parts: an encoder or HGNN-based feature extractor, SMOTE-based oversampling, edge generators, and a classifier.

4.1 Encoder

The encoder is a heterogeneous graph neural network based feature extractor [14] that generates learned embeddings for various node types within heterogeneous graphs. It is achieved through HGNN layers that aggregate content and embeddings from type-specific neighbors. The result is a set of node feature vectors in an embedding space, which are subsequently used for SMOTE oversampling.

Content Aggregation In this layer, the attribute matrix for each node is concatenated along the embedding dimension and then passed through a linear layer and nonlinearity to reduce its dimension back to the original embedding dimension, thereby combining the contextual information from all generated attributes of the nodes.

$$\mathcal{B}^p[u_k, :] = \sigma \left[\mathcal{W} \cdot \left(\bigoplus_{j=1}^n \mathcal{B}_j^p[u_k, :] \right) \right] \quad (1)$$

In this expression, $\mathcal{B}_j^p$ represents the j^{th} attribute matrix for node u_k of type p , n denotes the total number of attribute matrices, $\mathcal{B}^p$ is the content-aggregated matrix, $\mathcal{W}$ is the weight matrix for the linear layer, and σ represents an activation function such as ReLU.

Type-Specific Neighbor Aggregation In this layer, the content-aggregated representation matrix for each node type is combined with an aggregated feature matrix that captures information from neighboring node types, followed by dimensionality reduction through a linear layer. To begin, a list of frequently encountered neighboring nodes is extracted for each node along multiple random walk sequences. The top k most frequently appearing neighbors of each type are selected from these walks and assigned the same importance as direct edges, resulting in a total of t^2 adjacency matrices, where t is the number of node types. For each node, neighbor type-specific aggregation is achieved by using the appropriate adjacency matrix to retrieve embeddings of neighboring nodes from the feature embeddings and then averaging those embeddings. The resulting type-specific neighbor aggregated matrices $\mathcal{G}_s$ are later appended using attention weights mechanism. This final aggregated matrix is concatenated with the node's content-aggregated matrix and passed through a linear transformation to reduce dimensionality.

$$\mathcal{X}^p[u_i, :] = \mathcal{B}^p[u_i, :] \oplus \sum_p \beta_{u_i}^p \mathcal{G}^p[u_i, :] \quad (2)$$

$$\mathcal{G}^p[u_i, :] = \frac{1}{|\mathcal{N}^p(u_i)|} \sum_{u \in \mathcal{N}^p(u_i)} \sigma(\mathcal{W}_3 \cdot (\mathcal{B}^p[u, :])) \quad (3)$$

Here, $\mathcal{X}^p$ represents the final embedding for node u_i of type p , $\mathcal{N}^p$ denotes the set of type- p neighbors for the node, and $\beta_{u_i}^p$ are the attention weights used to combine the neighbor-aggregated matrices $\mathcal{G}_p$. The attention weights $\beta_{u_i}^p$ are defined as follows:

$$\beta_{u_i}^p = \frac{\exp(\sigma(\mathcal{W}_4 \cdot (\mathcal{G}^p[u_i, :] \oplus \mathcal{B}^p[u_i, :])))}{\sum_{u \in \mathcal{N}^p(u_i)} \exp(\sigma(\mathcal{W}_4 \cdot (\mathcal{G}^p[u_i, :] \oplus \mathcal{B}^p[u_i, :])))} \quad (4)$$

where σ is the LeakyReLU activation function.

4.2 Synthetic Minority Oversampling (SMOTE)

After mapping nodes into the embedding space, samples from the minority class in the training set are randomly selected to be oversampled using SMOTE. SMOTE operates by creating new samples from the minority class through interpolation between nearby neighbors of the same class within the embedding space. Let u_i be a labeled node from the minority class with label Y_{u_i} and let its nearest neighbor from the same class p be denoted by $\mathcal{X}_n^p(u_i)$ (n denotes nearest neighbour). This node is found by calculating the Euclidean distances between the nodes of class p and u_i , as shown:

$$\mathcal{X}_n^p(u_i) = \operatorname{argmin}_j \|\mathcal{X}^p[u_i, :] - \mathcal{X}^p[u_j, :]\|, \text{ s.t. } Y_{u_i} = Y_{u_j} \quad (5)$$

Using this closest neighbor, synthetic nodes are generated that retain the same label as the original nodes. The synthetic embedding $\mathcal{X}_s^p(u_i)$ (s implies

6 No Author Given

synthetic) is created by interpolating between the embeddings of u_i and its nearest neighbor, positioning it along the line connecting these nodes:

$$\mathcal{X}_s^p(u_i) = (1 - e) \cdot \mathcal{X}_n^p(u_i) + e \cdot \mathcal{X}^p[u_i, :] \quad (6)$$

where e is a randomly chosen scalar, with $e \in [0, 1]$. Synthetic nodes are generated iteratively until class imbalance is minimized. These new samples are appended to the original node feature matrix. The augmented adjacency matrices $\mathcal{A}_{pq}$ are inherited from the initial adjacency matrices $\mathcal{A}_{pq}^0$, where q represents any arbitrary node type. By employing an encoding layer prior to oversampling, GraphSMOTE minimizes noise in the generated samples. This is because embeddings of nodes within the same class show increased proximity with one another in the embedding space due to shared neighbor characteristics. This naturally clusters the source nodes and establishes clear boundaries between classes, allowing the oversampling process to generate new samples in these well-defined regions, thus enhancing the overall effectiveness of the technique.

4.3 Edge Generator

Since synthetic nodes are initially isolated in the graph, they may lack the necessary relational structure within their embeddings. A neural network model is introduced as an edge generator to create edges for these synthetic nodes to the original graph. It is trained on the edges of real nodes and predicts edges for the synthetic nodes. The goal of the edge generator is to reconstruct the adjacency matrix for real nodes based on their representations, which then enables accurate link predictions for the synthetic nodes. The newly predicted adjacency entries for each synthetic node are appended to the original matrix $\mathcal{A}_{pq}^0$. For simplicity, the generator employs a vanilla architecture. It uses weighted inner product of the embeddings of the relevant node types for edge generation.

$$\hat{\mathcal{A}}_{pq} = \sigma(\mathcal{X}^p \cdot \mathcal{W} \cdot \mathcal{X}^q) \odot \mathcal{A}_{pq} \quad (7)$$

where $\hat{\mathcal{A}}_{pq}$ is the predicted adjacency matrix between node types p and q in the augmented graph, and σ represents the sigmoid function. To enhance learning, the predicted matrix is element-wise multiplied ($\odot$) by the inherited adjacency matrix from the oversampling stage $\mathcal{A}_{pq}$ to prevent learning of irrelevant edges. Separate generators are trained to model edges between the target node type and other types. The loss function for the edge generator is given by:

$$L_e = \sum_u \|\hat{\mathcal{A}}'_{pq} - \mathcal{A}_{pq}^0\|^2 \quad (8)$$

where $\hat{\mathcal{A}}'_{pq}$ is the predicted adjacency matrix for the original graph, excluding synthetic nodes. Two methods [16] are implemented to incorporate edges for synthetic nodes in the augmented adjacency matrix. In the first approach, the generator is optimized solely for edge reconstruction, and edges for synthetic nodes are created by applying a threshold θ (set to 0.5):

$$\hat{\mathcal{A}}_{pq} = \begin{cases} 1, & \text{if } \hat{\mathcal{A}}_{pq} > \theta, \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

Here, the original edges for the real nodes from $\mathcal{A}_{pq}^0$ is restored after predictions. In the second approach, soft edges are introduced for synthetic nodes instead of binary values. In this case, gradients on $\hat{\mathcal{A}}_{pq}$ can flow back from the classifier, allowing the generator to be optimized using both the edge prediction loss and the node classification loss.

4.4 Classifier

A straightforward neural network alongside a heterogeneous neighbor aggregator layer for the target node type is utilized to derive freshly aggregated embeddings for the synthetic nodes, by incorporating relational information from the edges predicted in the edge generator step. This final embedding is then fed into an MLP layer for classification.

$$\mathcal{P}_{u_i} = \sigma(\text{MLP}(\mathcal{X}'[u_i, :])) \quad (10)$$

Here, $\mathcal{P}_{u_i}$ is the prediction for node u_i using updated embedding $\mathcal{X}'$, and σ denotes the softmax function. The classifier employs cross-entropy loss:

$$L_{\text{cls}} = \sum_{u_i \in U'} \sum_c [1(Y_{u_i} = c) \cdot \log(\mathcal{P}_{u_i})] \quad (11)$$

where Y_{u_i} is the actual class label for node u_i in the oversampled set U' , and $\mathcal{P}_{u_i}$ is the predicted probability for node u_i being in class c . During testing, the predicted class for each node u , denoted as Y'_u , is chosen as the class with the maximum predicted probability:

$$Y'_u = \arg \max_c \mathcal{P}_u. \quad (12)$$

5 Experiments

5.1 Optimization

The optimization goal in HetGSMOTE is to refine the weights for the three components: the feature extractor, edge generator, and classifier. The model uses two loss functions for training: the edge generation loss (L_e) from the edge generation task and the classification loss (L_{cls}) from the classifier. The combined loss function is defined as $L = \lambda \cdot L_e + L_{\text{cls}}$, where λ is a hyperparameter that decides their relative importance. HetGSMOTE's overall objective function mirrors that of GraphSMOTE, expressed as:

$$\min_{\mu, \nu, \gamma} (L_{\text{cls}} + \lambda \cdot L_e) \quad (13)$$

8 No Author Given

Here, μ , ν , and γ represent the parameters for the feature extractor, edge generator, and classifier. Additionally, pre-training the feature extractor and edge generator with L_e was tested to enhance model stability during training.

5.2 Dataset and Evaluation Metric

To demonstrate the versatility of our method, we used heterogeneous datasets of different sizes across various domains. These are AMiner (A-II) [14] and DBLP[12] for bibliographic datasets, IMDb[3] for movie network, and PubMed[13] for biomedical datasets.

For the imbalanced classification task, we adopt three evaluation metrics, consistent with those used in GraphSMOTE: accuracy (ACC), and mean F1 score (F1). The F1 score is averaged across all classes without weighting, allowing for a more accurate reflection of performance on minority classes. Additionally, the F1-score threshold is adjusted dynamically to achieve optimal results in the comparative analyses.

5.3 Experimental Settings:

To evaluate and compare the methods, experiments were conducted on baseline oversampling techniques and various configurations of HetGSMOTE. In the settings labeled as ‘T’ type, only the edge loss is applied for training or fine-tuning the edge generators. In contrast, the ‘O’ type settings incorporate both edge and classification loss for training or fine-tuning the edge generators. The baselines and specific experimental configurations are outlined below:

1. **Baseline 1: *no***- Original datasets without oversampling.
2. **Baseline 2: *up***- Basic upsampling by duplicating the source nodes to overcome the imbalance ratio. The inherited adjacency list for the synthetic nodes is:

$$\mathcal{A}_{pq}[u_i, :] = \mathcal{A}_{pq}^0[u, :] \quad (14)$$

where u is the source node for u_i and $\mathcal{A}_{pq}^0$, $\mathcal{A}_{pq}$ are the old and inherited adjacency matrices between node types p and q respectively.

3. **Baseline 3: *smote***- Oversampling using smote but on the raw embedding space, i.e., after the content aggregation step and without the edge generator. The inherited adjacency list for the synthetic node is given as:

$$\mathcal{A}_{pq}[u_i, :] = \min(1, \mathcal{A}_{pq}^0[u_1, :] + \mathcal{A}_{pq}^0[u_2, :]) \quad (15)$$

where u_1 and u_2 are the source nodes for u_i .

4. **Baseline 4: *reweight***- Cost-sensitive approach, where higher weight is added to the loss function component for minority classes, making the model more sensitive to their misclassification.
5. **Baseline 5: *em_sm***- SMOTE is applied after neighbor aggregation, excluding the edge generator step. This relies on the assumption that post-interpolation using SMOTE in the neighbor-aggregated embedding space, sufficient relational information is transferred to the synthetic node for classification. Hence, no augmented adjacency matrix is utilized.

6. **Setting 0: *gsmE***- Smote after neighbor aggregation and using the inherited adjacency matrix for the synthetic nodes without using the edge generator. This method shows the impact of the edge generator in HetGSMOTE.
7. **Setting 1: *gsmT***- A HetGSMOTE variant where the edge generator is trained exclusively on the edge loss.
8. **Setting 2: *gsmO***- A HetGSMOTE variant where the edge generator is trained using both the edge loss and classification loss.
9. **Setting 3: *preT***- In this HetGSMOTE variant, the edge generator and encoder undergo pre-training on the edge prediction task, followed by fine-tuning with the objective function in Equation (13). During fine-tuning, the edge generator is trained solely on the edge loss.
10. **Setting 4: *preO***- Similar to *preT*, but in this setting, the edge generator is fine-tuned using both edge loss and classification loss.

5.4 Configuration

All experiments were conducted on a 64-bit system with an Nvidia A100 GPU (80GB PCIe) and the ADAM optimizer. Three learning rates were tested, with $1e-4$ chosen for all datasets and models for optimal performance, alongside a weight decay of $5e-4$. The hyperparameter λ was set to $1e-6$ for *gsmO* and *preO* configurations, and 10 for others. Models were trained for 200 epochs, typically converging between 120 and 160 epochs. Embedding dimensions ranged from 128 to 256, based on attribute characteristics. To reduce bias, each baseline and configuration was tested 10 times on 10 training subsets, with predefined seeds ensuring reproducibility.

6 Results

We evaluated HetGSMOTE under various settings to analyze its impact on classification performance, learning stability, and complexity compared to baselines. Across all three metrics, HetGSMOTE variants consistently outperformed the baselines for imbalanced node classification as given in Table 5.4. Pre-trained settings (*preT* and *preO*) demonstrated superior performance over *gsmT*-*gsmO*, emphasizing the benefits of pre-training. While ‘O’ settings performed better with pre-training and exhibited lower standard deviation across metrics due to edge loss inclusion, ‘T’ settings showed better results without pre-training. Pre-trained settings also displayed greater generalization ability and stability compared to their non-pre-trained counterparts. Furthermore, HetGSMOTE’s integration of edge generators contributed to its superior performance over *gsmE* and *em_sm* methods. Experiments were conducted to evaluate HetGSMOTE’s performance across varying imbalance ratios, ranging from $[0.1, 0.9]$ in increments of 0.1, with oversampling applied until the class distribution was balanced. As shown in Table 5.4, HetGSMOTE consistently outperformed baselines across all datasets up to an imbalance ratio of 0.6, with improvements in ACC being most significant (approximately 1%) at extreme imbalance ratios of 0.1 and 0.2. For

10 No Author Given

Table 1. Test Accuracy and F1-Score vs Imbalanced Ratio at Different Settings - All Datasets (*Bold indicates the highest mean value across column, *italics* indicate proposed methods)

Dataset Setting		Accuracy (%)			F1 score (%)		
		0.2	0.4	0.6	0.2	0.4	0.6
AMiner	no	95.3 (0.7)	96.4 (1.2)	96.6 (1.4)	96.2 (0.8)	96.5 (0.9)	96.6 (1.2)
	up	95.9 (1.2)	96.7 (0.9)	96.7 (1.1)	96.4 (0.9)	96.4 (1.1)	96.6 (1.0)
	smote	95.8 (0.9)	96.7 (1.0)	96.5 (1.2)	96.3 (1.1)	96.4 (0.9)	96.7 (1.0)
	reweight	94.8 (1.8)	96.1 (0.9)	96.3 (1.3)	96.4 (0.8)	96.6 (0.8)	96.5 (1.1)
	em_sm	95.8 (0.8)	96.4 (1.1)	96.5 (1.2)	96.2 (0.8)	96.2 (0.9)	96.6 (1.5)
	gsmE	96.3 (0.9)	96.3 (1.5)	96.7 (0.9)	96.3 (0.9)	96.5 (1.3)	96.6 (0.9)
	<i>gsmT</i>	96.2 (1.4)	96.4 (1.2)	96.7 (1.1)	96.3 (1.0)	96.3 (0.8)	96.6 (1.1)
	<i>gsmO</i>	96.1 (1.3)	96.5 (1.0)	96.7 (1.2)	96.6 (0.7)	96.4 (0.8)	96.7 (1.1)
	<i>preT</i>	95.7 (0.9)	96.8 (1.3)	96.7 (1.0)	96.2 (0.8)	96.6 (1.0)	96.6 (1.0)
	<i>preO</i>	96.3 (1.2)	96.6 (1.3)	96.9 (1.1)	96.4 (1.0)	96.5 (0.8)	96.6 (1.0)
DBLP	no	69.2 (4.2)	76.9 (4.7)	80.1 (2.6)	79.7 (3.0)	82.8 (2.3)	84.1 (1.7)
	up	65.9 (5.8)	75.8 (4.1)	79.9 (2.8)	77.8 (4.6)	82.0 (2.8)	82.6 (2.9)
	smote	68.4 (5.9)	75.8 (3.1)	78.1 (4.9)	79.6 (4.2)	80.9 (2.6)	82.0 (3.2)
	reweight	68.9 (4.4)	78.1 (4.5)	82.1 (2.9)	78.3 (2.9)	82.3 (2.6)	84.3 (1.1)
	em_sm	69.8 (4.5)	73.4 (6.0)	78.1 (5.7)	79.2 (4.1)	80.0 (3.8)	82.1 (3.2)
	gsmE	66.8 (3.7)	77.2 (5.2)	78.1 (5.4)	77.7 (2.7)	82.8 (2.2)	81.9 (3.5)
	<i>gsmT</i>	70.4 (4.9)	78.6 (3.5)	79.5 (4.5)	80.4 (3.2)	83.2 (1.6)	82.6 (3.1)
	<i>gsmO</i>	67.7 (3.9)	75.3 (3.3)	82.2 (1.4)	78.2 (2.4)	82.1 (1.6)	84.0 (1.8)
	<i>preT</i>	69.3 (3.9)	77.5 (5.1)	81.5 (3.8)	78.4 (3.2)	81.9 (3.6)	83.3 (2.1)
	<i>preO</i>	70.7 (4.8)	78.0 (4.4)	80.1 (3.2)	79.5 (3.7)	83.3 (1.8)	82.7 (2.4)
PubMed	no	13.8 (1.1)	15.2 (1.4)	14.1 (1.2)	20.3 (2.4)	21.1 (1.6)	20.6 (1.5)
	up	14.7 (1.5)	15.0 (1.5)	14.6 (1.9)	19.8 (0.9)	21.1 (0.9)	20.5 (1.7)
	smote	15.9 (2.2)	14.9 (2.4)	15.6 (2.1)	21.7 (1.0)	20.5 (1.4)	19.9 (1.3)
	reweight	16.1 (1.7)	15.4 (1.4)	15.8 (1.7)	21.3 (1.0)	20.7 (2.0)	20.5 (1.3)
	em_sm	14.9 (1.5)	15.1 (1.2)	15.2 (1.2)	20.7 (1.8)	19.9 (1.4)	20.6 (1.3)
	gsmE	15.4 (2.1)	15.3 (1.3)	16.0 (1.2)	21.4 (1.4)	20.9 (0.9)	20.2 (1.3)
	<i>gsmT</i>	15.1 (1.3)	15.4 (1.8)	15.4 (2.1)	21.7 (1.7)	20.5 (1.2)	20.6 (1.2)
	<i>gsmO</i>	14.8 (1.1)	15.0 (1.9)	15.0 (1.9)	21.2 (1.2)	20.8 (1.6)	20.1 (1.5)
	<i>preT</i>	16.4 (2.4)	13.9 (1.5)	15.1 (1.2)	20.1 (1.6)	19.6 (2.0)	20.3 (1.2)
	<i>preO</i>	16.8 (1.9)	14.4 (1.7)	14.6 (1.1)	20.0 (1.4)	20.7 (1.2)	20.2 (1.4)
IMDb	no	45.2 (2.6)	48.7 (2.6)	50.1 (2.3)	53.1 (1.8)	53.9 (1.5)	54.8 (1.9)
	up	44.6 (2.4)	48.9 (2.7)	50.2 (2.5)	53.1 (1.7)	54.3 (1.1)	54.3 (1.8)
	smote	45.5 (2.4)	48.3 (2.4)	49.9 (1.7)	53.5 (1.9)	54.1 (1.4)	54.7 (1.7)
	reweight	43.9 (3.3)	46.6 (2.8)	48.8 (2.9)	53.3 (2.2)	54.1 (1.6)	54.2 (1.9)
	em_sm	45.7 (2.4)	47.5 (3.5)	49.4 (3.5)	53.2 (1.9)	54.2 (2.1)	54.6 (2.8)
	gsmE	44.2 (2.3)	47.9 (2.6)	50.1 (2.5)	53.1 (2.0)	54.2 (1.4)	54.7 (1.4)
	<i>gsmT</i>	43.8 (2.2)	47.5 (2.6)	49.2 (2.6)	52.9 (1.5)	53.6 (1.5)	54.4 (1.8)
	<i>gsmO</i>	43.3 (1.8)	47.1 (3.3)	49.2 (2.6)	52.5 (1.3)	54.1 (2.3)	54.6 (1.2)
	<i>preT</i>	45.7 (1.2)	48.6 (2.4)	50.8 (2.4)	53.8 (1.6)	53.4 (2.6)	54.8 (1.6)
	<i>preO</i>	44.8 (2.1)	49.5 (2.5)	50.3 (2.4)	53.2 (1.3)	54.2 (1.0)	54.8 (1.6)

higher imbalance ratios, the impact of oversampling diminishes as fewer synthetic nodes are generated, aligning with observations from prior work [16]. For the upsampling ratio experiments, HetGSMOTE variants maintained superior performance in ACC beyond the 0.6 upsampling ratio threshold. Based on all the experiments conducted, the variants of HetGSMOTE perform better than the baselines in 80% of all cases, with preO being the best performer among the variants followed closely by gsmT, showing that pretraining does better when the combined loss function is used for finetuning the edge generator.

7 Conclusion

The results highlight HetGSMOTE's strong performance on heterogeneous graphs, showcasing its adaptability across diverse datasets and its significant improvements over baseline methods. This study contributes to advancing HGNN-based methodologies in applications such as academic paper classification, offering practical strategies to manage class imbalance in complex, real-world datasets. Moving forward, we aim to expand our research in three main directions. First, we plan to explore HetGSMOTE on other tasks prone to class imbalance, such as link prediction, edge type classification, and node representation learning. Second, we will experiment with alternative base models, including HAN and MagNN, to assess our method's flexibility. Lastly, we intend to broaden our dataset selection to include more varied heterogeneous data, such as those based on reviews, to further validate the robustness of our approach. Future work can further explore the performance and versatility of the method through a systematic study of its application on various GNN architectures.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Chawla, N.V., Lazarevic, A., Hall, L.O., Bowyer, K.W.: Smoteboost: Improving prediction of the minority class in boosting. In: European Conference on Principles of Data Mining and Knowledge Discovery. pp. 107–119. Springer (2003)
2. Douzas, G., Bacao, F., Last, F.: Improving imbalanced learning through a heuristic oversampling method based on k-means and smote. *Information Sciences* **465**, 1–20 (Oct 2018). <https://doi.org/10.1016/j.ins.2018.06.056>, <http://dx.doi.org/10.1016/j.ins.2018.06.056>
3. Fu, X., Zhang, J., Meng, Z., King, I.: Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In: Proceedings of The Web Conference 2020. p. 2331–2341. WWW '20, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3366423.3380297>, <https://doi.org/10.1145/3366423.3380297>
4. Han, H., Wang, W.Y., Mao, B.H.: Borderline-smote: a new over-sampling method in imbalanced data sets learning. In: International Conference on Intelligent Computing. pp. 878–887. Springer Berlin Heidelberg (2005)

12 No Author Given

5. He, H., Bai, Y., Garcia, E.A., Li, S.: Adasyn: Adaptive synthetic sampling approach for imbalanced learning. In: 2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence). pp. 1322–1328. IEEE (2008)
6. Hsu, H.C., Lin, T.L., Wu, B.J., Hong, M.Y., Lin, C., Wang, C.Y.: Fincgan: A gan framework of imbalanced node classification on heterogeneous graph neural network. In: ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). pp. 5750–5754. IEEE (2024)
7. Kovács, G.: An empirical comparison and evaluation of minority oversampling techniques on a large number of imbalanced datasets. *Applied Soft Computing* **83**, 105662 (2019)
8. Liu, J., Shi, C., Yang, C., Lu, Z., Philip, S.Y.: A survey on heterogeneous information network based recommender systems: Concepts, methods, applications and resources. *AI Open* **3**, 40–57 (2022)
9. Ma, Y., Tian, Y., Moniz, N., Chawla, N.V.: Class-imbalanced learning on graphs: A survey. *arXiv preprint arXiv:2304.04300* (2023)
10. Shi, C.: *Heterogeneous Graph Neural Networks*. Springer Nature Singapore, Singapore (2022)
11. Wang, H., Wang, J., Wang, J., Zhao, M., Zhang, W., Zhang, F., Xie, X., Guo, M.: Graphgan: graph representation learning with generative adversarial nets. In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence. AAAI'18/IAAI'18/EAAI'18*, AAAI Press (2018)
12. Wang, X., Ji, H., Shi, C., Wang, B., Ye, Y., Cui, P., Yu, P.S.: Heterogeneous graph attention network. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. p. 2022–2032. WWW '19, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3308558.3313562>, <https://doi.org/10.1145/3308558.3313562>
13. Yang, C., Xiao, Y., Zhang, Y., Sun, Y., Han, J.: Heterogeneous network representation learning: A unified framework with survey and benchmark. *IEEE Transactions on Knowledge and Data Engineering* **34**(10), 4854–4873 (2020), <https://github.com/yangji9181/HNE/tree/master/Data>
14. Zhang, C., Song, D., Huang, C., Swami, A., Chawla, N.V.: Heterogeneous graph neural network. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. p. 793–803. KDD '19, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3292500.3330961>, <https://doi.org/10.1145/3292500.3330961>
15. Zhang, Z., Cui, P., Zhu, W.: Deep learning on graphs: A survey. *IEEE Transactions on Knowledge & Data Engineering* **34**(01), 249–270 (jan 2022). <https://doi.org/10.1109/TKDE.2020.2981333>
16. Zhao, T., Zhang, X., Wang, S.: Graphsmote: Imbalanced node classification on graphs with graph neural networks. In: *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. p. 833–841. WSDM '21, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3437963.3441720>, <https://doi.org/10.1145/3437963.3441720>